



New Mansoura University
Faculty of Computer Science and Engineering



Advanced Machine Learning

Ensemble Learning and Random Forests

Islam Reda

Associate Professor

islam.reda@nmu.edu.eg

Outline

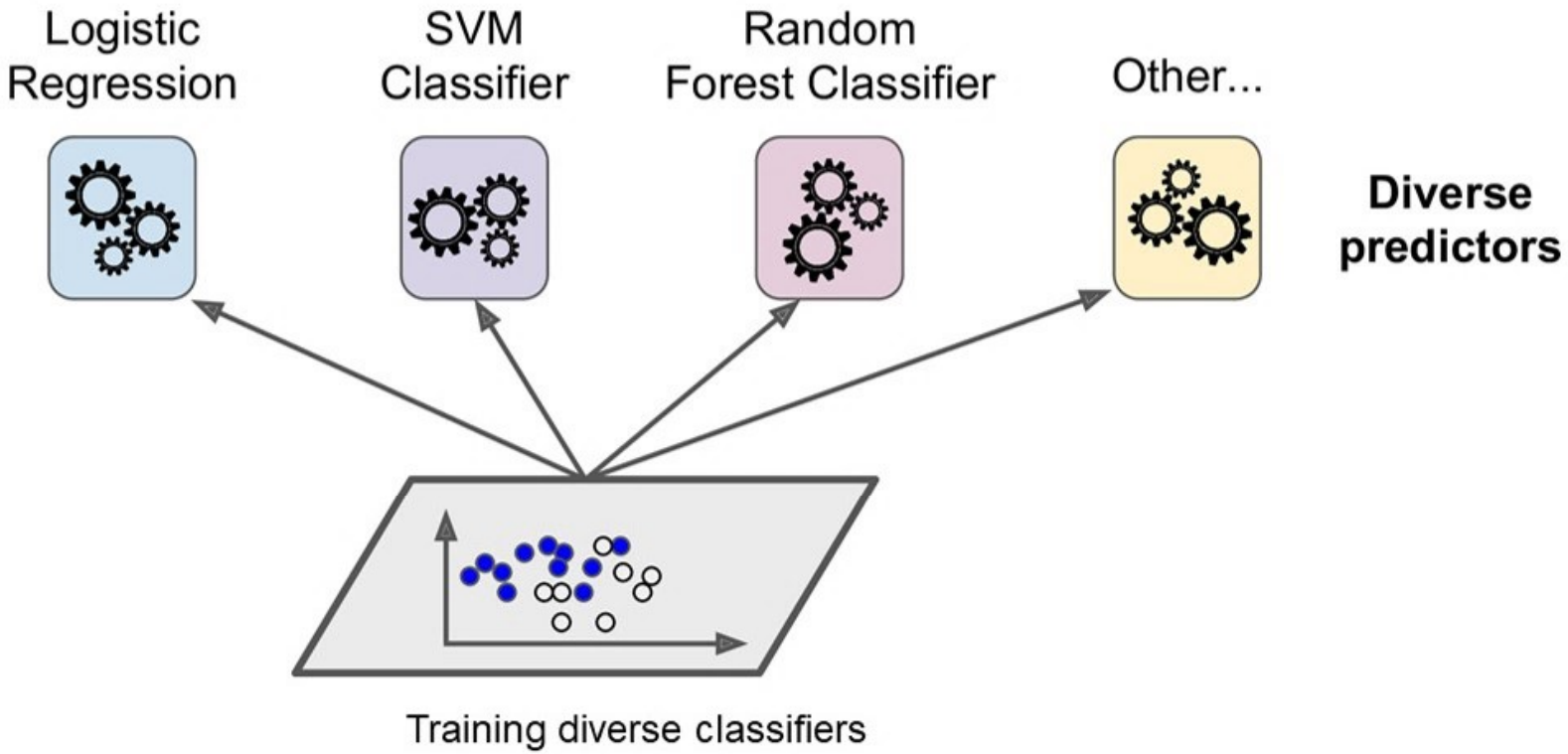
- Voting Classifiers
- Bagging and Pasting
  - Out-of-Bag Evaluation
- Random Patches and Random Subspaces
- Random Forests
  - Extra-Trees
- Boosting
- Stacking

Basic Idea

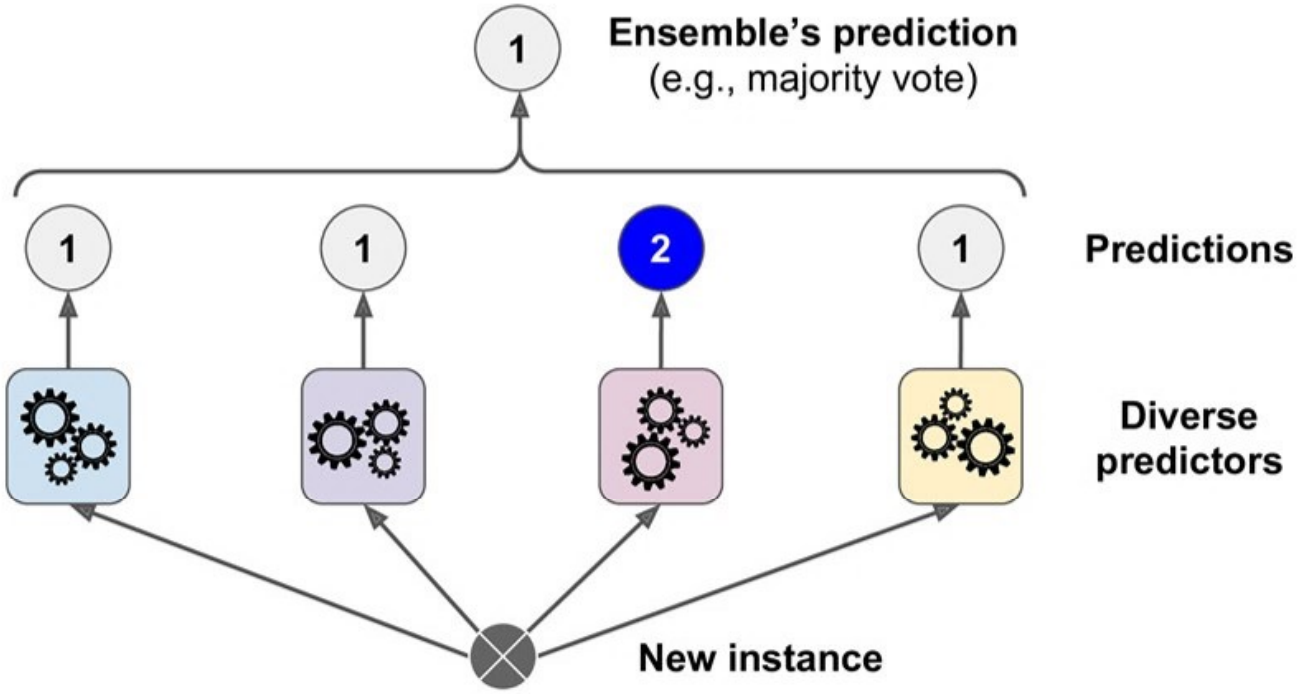
- If you aggregate the **predictions of a group of predictors** (such as classifiers or regressors), you will often get **better predictions** than with the best individual predictor. A group of predictors is called an **ensemble**; thus, this technique is called **ensemble learning**.
- For example, you can **train a group of Decision Tree classifiers**, each on a different random subset of the training set. To make predictions, you just obtain the predictions of all individual trees, then **predict the class that gets the most votes**. Such an ensemble of Decision Trees is called a **Random Forest**.

Voting Classifiers

Voting Classifiers



Voting Classifiers



Hard voting classifier predictions

Voting Classifiers

- The **majority-vote** classifier is called a **hard voting classifier**.
- This voting classifier often achieves a higher accuracy than the best classifier in the ensemble. Even if each classifier is a **weak learner** (meaning it does only slightly better than random guessing), the **ensemble** can still be a **strong learner** (achieving high accuracy), provided there are a **sufficient number of weak learners** and they are **sufficiently diverse**.
- One way to get **diverse classifiers** is to train them using very **different algorithms**. This increases the chance that they will make very different types of errors, improving the ensemble's accuracy.

Voting Classifiers

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC

log_clf = LogisticRegression()
rnd_clf = RandomForestClassifier()
svm_clf = SVC()

voting_clf = VotingClassifier(
        estimators=[('lr', log_clf), ('rf', rnd_clf), ('svc', svm_clf)],
        voting='hard')
voting_clf.fit(X_train, y_train)
```

Voting Classifiers

```
>>> from sklearn.metrics import accuracy_score
>>> for clf in (log_clf, rnd_clf, svm_clf, voting_clf):
...     clf.fit(X_train, y_train)
...     y_pred = clf.predict(X_test)
...     print(clf.__class__.__name__, accuracy_score(y_test, y_pred))
...
```

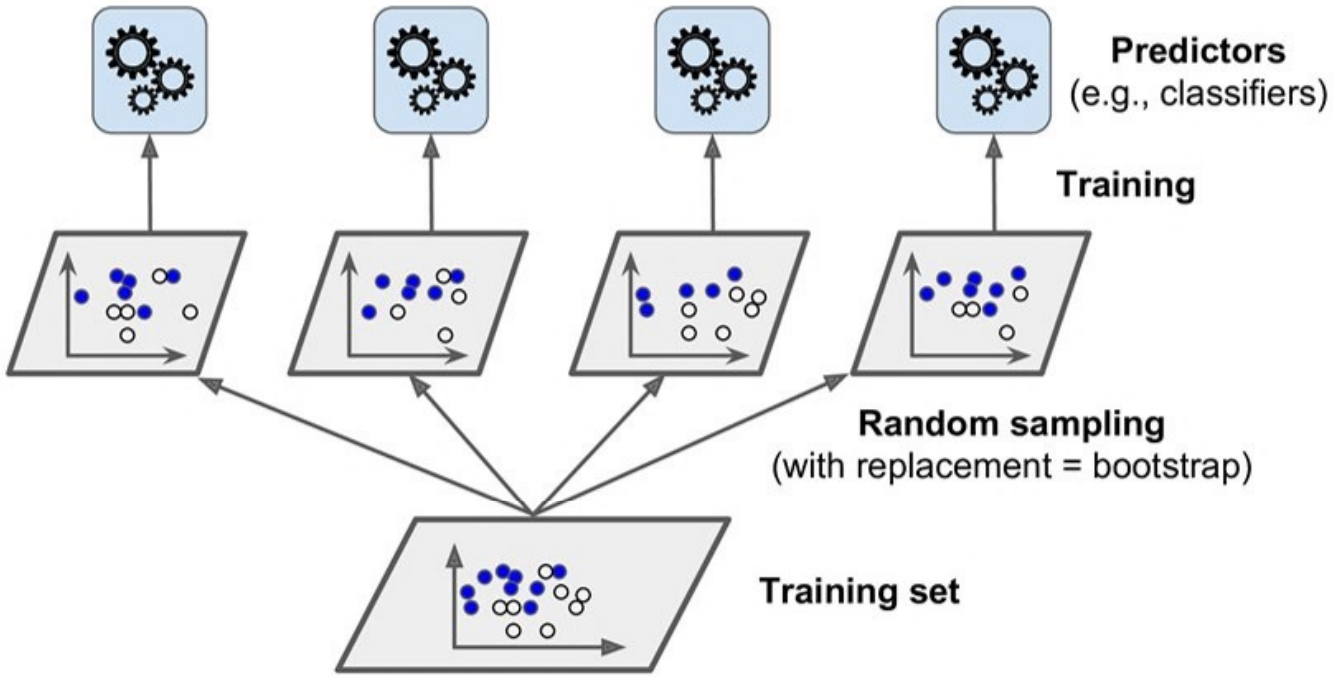
LogisticRegression 0.864
RandomForestClassifier 0.896
SVC 0.888
VotingClassifier 0.904

Voting Classifiers

- If **all classifiers** are able to **estimate class probabilities** (they have a predict_proba() method), then you can tell Scikit-Learn to predict the class with the highest class probability, averaged over all the individual classifiers. This is called **soft voting**.
- It often achieves **higher performance than hard voting** because it **gives more weight to highly confident votes**. All you need to do is replace **voting="hard"** with **voting="soft"** and ensure that all classifiers can estimate class probabilities.
- In case of the SVC class, you need to set its **probability** hyperparameter to **True**.

Bagging and Pasting

Bagging and Pasting



Pasting/bagging training set sampling and training

Bagging and Pasting

- Another approach to get a **diverse set of classifiers** is to use the same training algorithm for every predictor, but to **train them on different random subsets of the training set**.
- When **sampling** is performed **with replacement**, this method is called **bagging** (short for bootstrap aggregating). When **sampling** is performed **without replacement**, it is called **pasting**. Bagging allows training instances to be sampled several times for the same predictor.
- Once all predictors are trained, the ensemble can make a prediction for a new instance by simply **aggregating the predictions** of all predictors.

Bagging and Pasting

- The **aggregation function** is typically the **statistical mode** (i.e., the most frequent prediction, just like a hard voting classifier) **for classification**, or the **average for regression**.
- Predictors can all be **trained in parallel, via different CPU cores or even different servers**. Similarly, predictions can be made in parallel.
- This is one of the reasons why **bagging and pasting** are such popular methods: they **scale very well**.

Bagging and Pasting in Scikit-Learn

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

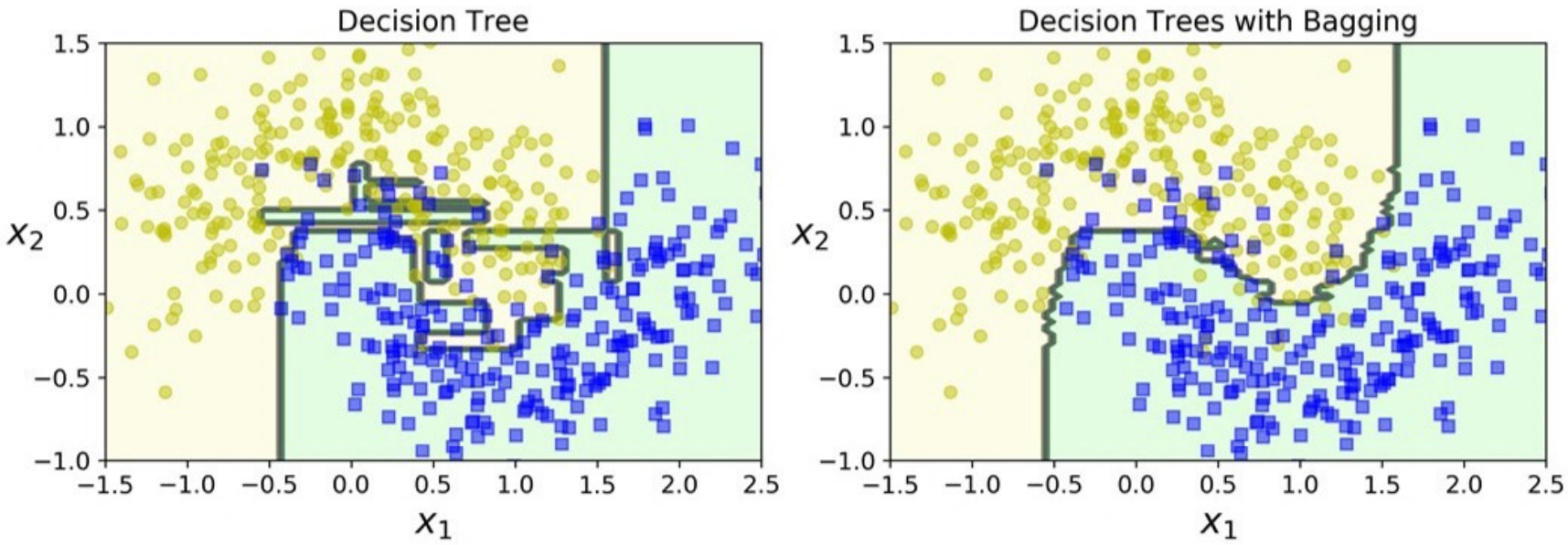
bag_clf = BaggingClassifier(
        DecisionTreeClassifier(), n_estimators=500,
        max_samples=100, bootstrap=True, n_jobs=-1)
bag_clf.fit(X_train, y_train)
y_pred = bag_clf.predict(X_test)
```

- This code is an example of **bagging**, but if you want to use **pasting** instead, just **set bootstrap=False**).
- The **n_jobs** parameter tells Scikit-Learn the **number of CPU cores** to use for training and predictions (–1 tells Scikit-Learn to use all available cores).

Bagging and Pasting

- The **BaggingClassifier automatically performs soft voting** instead of hard voting if the base classifier can estimate class probabilities (i.e., if it has a predict_proba() method), which is the case with Decision Trees classifiers.
- The **ensemble's predictions** will likely **generalize** much **better** than the single Decision Tree's predictions.
- Overall, bagging often results in better models than pasting, which explains why it is generally preferred.

Bagging and Pasting



A single Decision Tree versus a bagging ensemble of 500 trees

Out-of-Bag Evaluation

Out-of-Bag Evaluation

- With bagging, some instances may be sampled several times for any given predictor, while others may not be sampled at all.
- **By default a BaggingClassifier samples m training instances with replacement** (bootstrap=True), where m is the size of the training set.
- Only about **63%** of the training instances **are sampled on average** for each predictor. The **remaining 37%** of the training instances that are not sampled **are called out-of-bag (oob)** instances. Note that they are not the same 37% for all predictors.
- A predictor can be evaluated on these instances, without the need for a separate validation set.

Out-of-Bag Evaluation

- You can **evaluate the ensemble** itself **by averaging out the oob** evaluations of each predictor.
- In Scikit-Learn, you can set **oob_score=True** when creating a BaggingClassifier to request an automatic oob evaluation after training.

```
>>> bag_clf = BaggingClassifier(
...         DecisionTreeClassifier(), n_estimators=500,
...         bootstrap=True, n_jobs=-1, oob_score=True)
...
>>> bag_clf.fit(X_train, y_train)

>>> bag_clf.oob_score_
0.90133333333333332
```

Out-of-Bag Evaluation

- According to this oob evaluation, this BaggingClassifier is likely to achieve about 90.1% accuracy on the test set. Let's verify this:

```
>>> from sklearn.metrics import accuracy_score
>>> y_pred = bag_clf.predict(X_test)
>>> accuracy_score(y_test, y_pred)
0.91200000000000003
```

We get 91.2% accuracy on the test set—close enough!

Random Patches and Random Subspaces

Random Patches and Random Subspaces

- The BaggingClassifier class supports **sampling the features** as well.
- This is controlled by two hyperparameters: **max_features** and **bootstrap_features**. They work the same way as max_samples and bootstrap, but for feature sampling instead of instance sampling.
- This is **useful** when dealing with **high-dimensional inputs** (such as images).

Random Patches and Random Subspaces

- **Sampling both training instances and features** is called the **Random Patches** method.
- Keeping all training instances (i.e., bootstrap=False and max_samples=1.0) but **sampling features** (i.e., bootstrap_features=True and/or max_features smaller than 1.0) is called the **Random Subspaces** method.
- Sampling features results in even more predictor diversity.

Thank You